

Products, Purchases

& Sales for Stockflow

A working study of inFlow Inventory's core workflows — annotated with real screens from its Reports, Sales, Purchasing and Products areas — translated into a build spec for a leaner, faster, light/dark-ready inventory app.

Reference product inFlow Inventory (app.inflowinventory.com)

Modules in scope Products · Purchases · Sales **Prepared** 2 Sep 2026

01 — Foundation

Purpose & scope

This document reverse-engineers the three modules that matter most in a small-business inventory tool, so they can be rebuilt — deliberately simpler where inFlow is dense, and more modern where it looks its age.

inFlow organizes itself around three connected records: a **Product** sits in the middle, a **Purchase order** brings stock in from a vendor, and a **Sales order** sends stock out to a customer. Every quantity shown anywhere in the app traces back to one of these three. That structure is worth keeping — it's the reason the app feels coherent even though it does a lot.

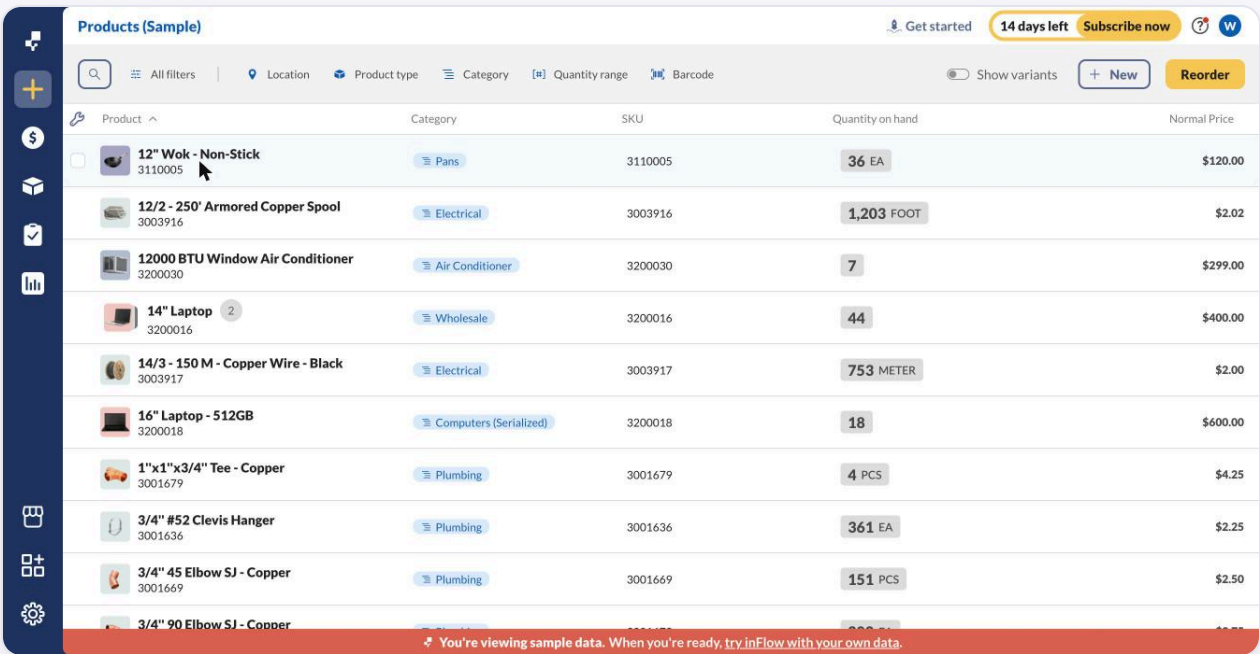
What follows is organized the same way: one section per module, each with the fields observed, the states a record moves through, and what to change when we build our own version. A shared section after the three modules covers mechanics that show up everywhere — statuses, filters, multi-location stock, custom fields. The last two sections turn all of it into a data model and a phased build order.

Design principle carried through this whole spec: match inFlow's information density (it earns its complexity — nothing here is decoration) but replace its dated, mouse-heavy chrome with a faster, keyboard-friendly UI that also works in dark mode, which inFlow does not offer at all.

02 — Module

Products

The catalog. Every product is one SKU with its own identity, pricing, physical attributes and vendor list — and a live, per-location stock count that updates from every purchase and sale.



OBSERVED Products list — filterable by location, product type, category, quantity range and barcode; a "Show variants" toggle expands sized/colored variants inline.

FIELDS CAPTURED ON A PRODUCT RECORD

FIELD	NOTES
Name & SKU	SKU is the durable identifier; name is freely editable without breaking history.

FIELD

NOTES

Category	Single-select, used for filtering and reporting — not a full taxonomy tree.
Barcode	Separate from SKU; scannable for stockroom lookups and label printing.
Description	Free text, shown to staff only — not customer-facing by default.
Dimensions & weight	L × W × H plus weight; feeds shipping/freight estimates.
Unit of measure	Stocking unit (e.g. "ea") with an alternate purchasing unit ("1 case = 5 ea") — quantities auto-convert.
Attributes	Brand, color, material, country of origin — plus custom fields per business (Amps, WHMIS #, QA Passed?).
Pricing schemes	Multiple named price tiers (Normal, wholesale, per-customer-group), each as markup % or fixed price over cost.
Cost	Editable manually or computed (FIFO/average) — drives margin on every order line.
Quantity on hand	Rolled up across locations; breaks down per-warehouse with a mini bar chart.

The screenshot displays the inFlow software interface. On the left is a dark sidebar with navigation icons. The main area is titled 'Products (Sample)' and shows a list of products on the left and a detailed view of the '12" Wok - Non-Stick' on the right. The product detail view includes an image of the wok, its SKU (3110005), a description, and various attributes like dimensions, brand, color, and material. Below the attributes are tabs for 'Overview', 'Product vendors', 'Order history', and 'Movement history'. A red banner at the bottom states: 'You're viewing sample data. When you're ready, try inFlow with your own data.'

OBSERVED Product detail — image, identity fields and custom attributes up top; stock-by-location and pricing side by side below; tabs for Overview, Product vendors, Order history, Movement history.

Sub-tabs worth keeping

What to simplify

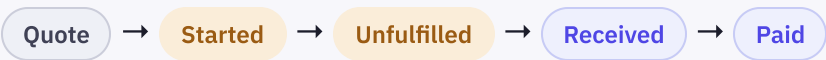
03 – Module

Stock coming in. A purchase order records what was ordered from a vendor, tracks partial receiving against it, and settles a bill.

OBSERVED Purchase orders list — tabbed by lifecycle stage (Quote / Open / Unpaid / All) with a live count badge on the active tab; filterable by location, vendor, status and date.

Status lifecycle

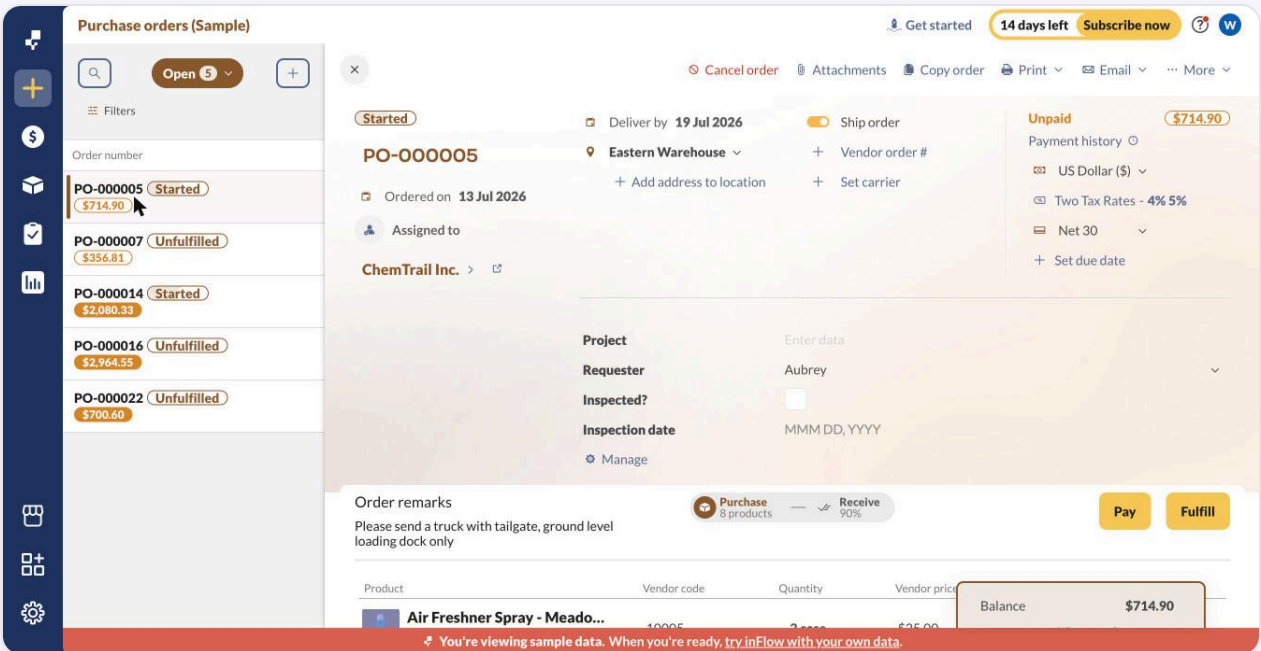
A purchase order moves through a fixed sequence, each stage gating what actions are available:



Two progress trackers run independently on the same order — **Receive** (how much of the ordered quantity has arrived) and **Pay** (how much of the bill is settled). An order can be 100% received and 0% paid, or vice versa; the UI shows both as separate progress rings so neither state hides the other.

HEADER FIELDS

Vendor	Linked record; carries default lead time and payment terms.
Order / deliver-by dates	Deliver-by drives the reorder & forecasting views.
Ship-to location	Which warehouse receives the stock.
Payment terms	Net 30 etc., plus per-order tax rate override.
Custom fields	Project, Requester, Inspected?, Inspection date — added per business.



OBSERVED Purchase order detail — vendor panel, custom fields block, dual Purchase/Receive progress indicator, and a line-item table with vendor code, quantity and vendor price columns.

Line-item mechanics

Each line carries the vendor's own product code and price (which can differ from the catalog's default cost), an ordered quantity, and a running received quantity — so a line can be partially received across several deliveries without splitting the order. Receiving a line writes a stock movement and updates the product's on-hand quantity at that location immediately.

What to change

Keep the dual receive/pay tracking — it's the one genuinely load-bearing idea here. Drop the "Vendor order #" and "Set carrier" micro-fields into an optional shipping panel instead of the main header, and let receiving happen against a barcode scan rather than manual quantity entry as the default path.

04 – Module

Sales

Stock going out. Structurally a mirror of Purchases — a customer instead of a vendor, fulfillment instead of receiving — but with its own quoting and invoicing path.

Sales orders (Sample)

Get started

14 days left

Subscribe now

?

W

Quote

Open 9

Ready to Ship 0

Unpaid 4

+ New sales order

All filters

Location

Status

Customer

Order date

Ship date

Due date

PO #

Assigned to

Order number	Customer	Order date	Requested ship d...	Location	Total	Balance
SO-000025 Fulfilled	Production Planet	T 2 Sep 2026		Eastern Warehouse	\$282.43	\$282.43
SO-000024 Unfulfilled	Beth May	T 31 Aug 2026		Eastern Warehouse	\$162.22	\$162.22
SO-000023 Unfulfilled	Anom Inc.	T 29 Aug 2026		Eastern Warehouse	\$400.00	\$400.00
SO-000017 Unfulfilled	Truck 34	28 Aug 2026		Truck 34 (Eastern)	\$54.25	\$54.25
SO-000022 Unfulfilled	Bollinger Machine & Shipyard	25 Aug 2026		Eastern Warehouse	\$124.25	\$124.25
SO-000021 Unfulfilled	City of Newburg	T 24 Aug 2026		Eastern Warehouse	\$665.99	\$665.99
SO-000014 Fulfilled	Project #841025	23 Aug 2026		Site 11 (Eastern)	\$66.00	\$66.00
SO-000018 Fulfilled	Bollinger Machine & Shipyard	22 Aug 2026		Eastern Warehouse	\$280.00	\$280.00
SO-000016 Started	Truck 34	20 Aug 2026		Eastern Warehouse	\$148.50	\$148.50

You're viewing sample data. When you're ready, try inFlow with your own data.

OBSERVED Sales orders list — same tabbed-status pattern as Purchases (Quote / Open / Ready to Ship / Unpaid / All), plus a per-row courier/tracking icon.

Status lifecycle

Mirrors purchases with fulfillment-specific language:



Here the twin trackers are **Pick** (percentage of ordered units pulled from shelf) and payment (**Mark paid**). A "Ship order" toggle and a separate "Same as billing address" toggle control whether the order needs its own shipping flow at all — useful for the in-store-pickup or B2B-drop-ship cases where it doesn't.

HEADER FIELDS

Customer	Linked record; billing and shipping address default from it, editable per order.
Fulfill-from location	Which warehouse the order ships from.
Sales rep	Assignable; used for commission and workload reporting.
Tax & payment terms	Tax rate resolved from ship-to region; terms default from customer.
P.O. #	The customer's own purchase-order reference, stored for their paperwork.

Sales orders (Sample)

Get started

14 days left

Subscribe now

?

W

+

\$

📦

✓

📄

⚙️

Filters

Order number

SO-000025

Fulfilled

\$282.43

SO-000024

Unfulfilled

\$162.22

SO-000023

Unfulfilled

\$400.00

SO-000017

Unfulfilled

\$54.25

SO-000022

Unfulfilled

\$124.25

SO-000021

Unfulfilled

\$665.99

SO-000014

Fulfilled

\$66.00

SO-000018

Fulfilled

\$280.00

SO-000016

Started

\$148.50

Fulfilled

SO-000025

Fulfill from Eastern ...

Ordered on 2 Sep 2026

Assigned to

+ P.O. #

Sales rep

Production Planet

Graciela Ruta

111-222-3339

Not-A-Person@email-address.com

Billing and shipping address

2371 Address Blvd

Chagrin Falls, OH 12351

United States

+ Add custom fields

+ Add order remarks

Order

1 product

Pick

100%

Mark paid

Unfulfill

Product	Quantity	Unit price	Subtotal
Consultant Senior - Hourly	4 hr	\$65.00	\$260.00
320584			
Balance			\$282.43

You're viewing sample data. When you're ready, try inFlow with your own data.

OBSERVED Sales order detail — customer & address panel, invoicing terms in the right rail, Order/Pick progress pair, and a "Mark paid" primary action once fulfillment is underway.

What to change

inFlow keeps quotes and orders as separate list items sharing one screen — worth keeping as-is, since converting a quote to an order without re-entering data is genuinely useful. The one gap: no visible margin/profitability indicator on the order itself, even though cost is known per line. Surface it — a small margin % next to the total is nearly free once cost and price both exist on the line.

05 — Shared mechanics

Cross-cutting mechanics

Patterns that appear identically in all three modules — build these once, as shared components, rather than per-module.

Navigation

Status-tab lists

Every list screen (Products excepted) opens on a tab strip — Quote / Open / Unpaid / All — with a live count badge. Tabs are just saved filters; build one filter engine and expose common views as tabs.

Structure

List + detail split view

Clicking a row opens the record beside the list, not over it — the list stays scrollable and selectable. Preserve this on desktop; collapse to full-screen detail below ~900px.

Data model

Multi-location stock

Every quantity is location-scoped first, summed second. Never store a single "quantity" field on a product — always (product, location) → quantity, with the total as a derived view.

Extensibility

Custom fields

Defined once in settings, then available on products, orders, customers and vendors. This is how inFlow supports wildly different businesses (a wok seller and an electrical wholesaler) from one schema.

Audit

Movement / order history

Every stock-affecting action (sale, purchase, transfer, adjustment) writes an immutable movement row. The "current quantity" is always a sum over movements, never a value that gets overwritten in place.

Trust

Independent progress pairs

Fulfillment/receiving progress and payment progress are tracked as two separate percentages on every order. Never collapse them into one status — "shipped" and "paid" are different facts.

Multi-tenant

Tenant isolation

Every business table carries an organization id, enforced centrally (one query-scoping mechanism, not a convention every route has to remember) rather than per feature — the one piece of this system that's genuinely expensive to retrofit once real customer data exists.

Visual design system

inFlow's UI is functional but visibly dated — dense toolbars, no dark mode, dashboard-orange used for both branding and status color. The direction below keeps the density where it earns its keep and modernizes everything else.

Palette — indigo / cool slate

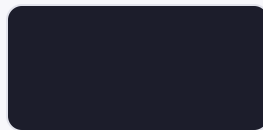
A cool, software-native base (blue-tinted greys, not inFlow's warm grey or a green-leaning neutral) with a single indigo accent reserved for primary actions, links and focus states. Status color stays on its own amber/red/green scale so it never competes with the brand hue — the two systems are read differently on purpose: accent means "act on this," status means "this is the state of that."



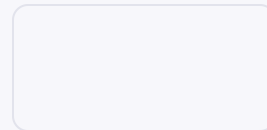
Accent (light) — Indigo 600
#4f46e5



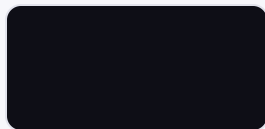
Accent (dark) — Indigo 400
#818cf8



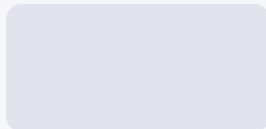
Ink (light mode)
#1c1d2b



Bg (light)
#f7f7fb



Bg (dark)
#0e0e16



Line (light)
#e2e3ec



Line (dark)
#2a2b38



Status — Amber
#9a5b13

Why the accent shifts shade between modes: #4f46e5 on white and #818cf8 on near-black hit roughly the same perceived contrast. Reusing #4f46e5 as-is on a dark background would sit closer to the surface color than to the text — it would read as muted instead of primary. Dark mode always needs a lighter, more saturated step of the same hue to do the same job.

Type

Fraunces for headings — a warm serif with enough weight to anchor a data-dense page without feeling corporate. **IBM Plex Sans** for UI and body text — built for interfaces, holds up at small sizes. **IBM Plex Mono** for anything tabular or identifier-like: SKUs, order numbers, prices, dates — so numbers align and read as data, not prose.

Dark mode is not optional

inFlow has none. Build both palettes from the same token set from day one — every color a component uses (surface, ink, line, accent, status) resolves through a token, never a literal hex, so light and dark stay in lockstep as the product grows. Respect the OS-level "system" preference by default, with an explicit override in settings.

Density & layout

Keep the list+detail split view and the tabbed status filters — they're the reason inFlow feels fast once learned. Modernize the chrome: replace inFlow's small hit-targets and grey-on-grey icon buttons with a consistent 40px control height, visible focus rings, and command-palette (⌘K) search across products, orders and customers as a keyboard-first alternative to hunting through the sidebar.

07 — Schema

Data model

The minimum table set implied by the three modules above. Everything else in the app — reports, reordering, dashboards — is a view over these.

```
-- tenancy — every table below also carries organization_id→organization, --
omitted per line below for readability; see the roadmap's Week 18-19
organization (id, name, slug, created_at) membership (id, user_id→user,
organization_id→organization, role) -- OWNER/ADMIN/STOREKEEPER/STAFF --
catalog product (id, sku, name, category_id, barcode, cost, unit,
description, attrs jsonb) product_price (id, product_id→product,
scheme_name, price) vendor_product (id, product_id→product,
vendor_id→vendor, vendor_sku, vendor_price, lead_time_days) -- parties
vendor (id, name, terms, contact jsonb) customer (id, name, terms,
billing_address, shipping_address) -- stock (location-scoped, always derived
from movements) location (id, name) stock_movement (id, product_id→product,
location_id→location, qty_delta, reason, source_id, created_at) -- purchases
purchase_order (id, number, vendor_id→vendor, location_id→location, status,
order_date, deliver_by, terms) purchase_order_line (id,
order_id→purchase_order, product_id→product, qty_ordered, qty_received,
```

```
unit_cost) -- sales sales_order (id, number, customer_id→customer,  
location_id→location, status, order_date, ship_by, terms) sales_order_line  
(id, order_id→sales_order, product_id→product, qty_ordered, qty_picked,  
unit_price) -- payments, shared shape across both order types payment (id,  
order_type, order_id, amount, paid_at, method) -- background jobs & pre-  
aggregated analytics job (id, type, status, payload jsonb, result jsonb,  
created_at) -- PENDING/RUNNING/DONE/FAILED daily_stock_summary (id,  
product_id→product, day, qty_in, qty_out) daily_sales_summary (id, day,  
revenue_cents, order_count)
```

Why derive quantity from movements: every screenshot above shows a quantity that had to reconcile perfectly across a product page, an order line, and a location breakdown. The only way to guarantee that without triple-writing is one source of truth — the movement ledger — with every other number computed from it.

Why organization_id belongs on the row, not just the query: a tenant filter that lives only in application code is one forgotten WHERE clause away from leaking company A's stock levels into company B's dashboard. Putting the column on every tenant-scoped table and enforcing it centrally — a Prisma Client Extension in this build, a Postgres Row-Level Security policy in others — makes the leak structurally hard instead of relying on every route remembering.

08 — Sequencing

Build roadmap

Phases 1-4 are shipped and live. Phase 5 is the current 8-week push: turning the single-tenant build into a real multi-company product, with the Sales module and the design-system rollout riding along. The target bar is inFlow's actual daily workflow done better — not its entire feature catalog (no barcode/label printing, multi-currency, B2B showroom, or POS in this phase).

Shipped

Phases 1-4

Products, stock, purchasing & reporting

- Product/category CRUD, multi-location stock, race-safe stock-in/out/transfer with a full movement audit trail
- Supplier + purchase-order model with line items and a receiving flow that auto-creates stock-in transactions
- Dashboard analytics, CSV import/export, low-stock alerting, deployed to Vercel + Neon Postgres

Wk 18

Tenancy model

Multi-tenancy data model

- Organization + Membership (role: OWNER/ADMIN/STOREKEEPER/STAFF), replacing the old global user role
- `organization_id` backfilled onto every business table via a data migration, all existing users attached to one default org
- A Prisma Client Extension that auto-injects the org filter into every tenant-scoped query — the "can't forget it" mechanism

Wk 19

Hardening

Org switcher, RBAC & tenancy tests

- Org switcher in the nav; a Team admin panel for inviting members and assigning roles
- Pooled DB connection string and a real cron for the low-stock alert job (both cheap, both overdue)
- A Postgres CI integration suite whose whole job is proving org A can never read or mutate org B's data

Wk 20-21

Sales

Sales module, born org-scoped

- Customer + sales-order + line-item models, mirroring Purchases' shape exactly
- A race-safe fulfillment endpoint reusing the same conditional-update pattern that already prevents overselling on the stock side
- Customer/sales-order UI, per-order margin display, revenue & top-customer dashboard cards

Wk 22

Scale-out

Background jobs & pre-aggregated analytics

- A DB-backed job queue (no new infrastructure) so CSV import/export can't hit a function timeout at real volume

- Daily rollup tables for stock and sales, so dashboard analytics stop re-scanning raw transaction history as it grows

Wk 23

Design system

Dark mode, full rollout

- Tokens, theme toggle, sidebar highlight, and the account menu are already live — this finishes migrating every remaining screen onto them
- A contrast re-check against both themes with the final token values

Wk 24-25

Harden & ship

Security audit & scale-readiness review

- An org-scoping audit across every route, rate limiting on auth and invites, error boundaries with structured logging
- Full regression pass, then an honest "known scale ceiling" note in the README — caching, hot-row handling, dedicated search, and multi-region are deliberately deferred until real traffic asks for them, not built speculatively now

Compiled from a live walkthrough of app.inflowinventory.com (sample data) — Reports, Sales orders, Purchase orders and Products areas. For internal planning use.